FlashInfer MLSys 2026 GDN Kernel - Team Kachua

Pushkar Patel (thepushkarp@gmail.com) and Romit Jain (romit.73@gmail.com)

May 1, 2026

1 Solution Overview

Gated Delta Networks (GDNs) are recurrent sequence models that maintain a per-head state across tokens. At each token, the kernel applies a gated decay to the previous state, performs a key/value update, and projects the query through the updated state to produce an output. The competition evaluates this operator in two regimes: decode and prefill. Decode corresponds to single-token generation. Prefill processes prompt sequences before generation begins.

The final implementation uses CUDA for decode and Triton for prefill. In decode, most of the improvement came from changing the work decomposition. Instead of assigning one warp to each V row, the final kernel uses one warp to process multiple V rows, which amortizes Q/K/gate handling, indexing, and reduction setup across rows. This reduced repeated work and improved instruction reuse in the single-token decode path.

In prefill, most of the improvement came from shape-specialized dispatch. The final Triton implementation selects among execution paths based on the ragged batch shape: a fused chunkwise path for smaller cases, a two-kernel path that precomputes reusable chunk-local terms for long low-batch workloads, and a flattened many-sequence path that exposes more independent work for large ragged batches. Overall, CUDA was better suited to decode because it gave direct control over the compact one-token instruction stream, while Triton was better suited to prefill because the chunkwise formulation maps naturally to tensor-core-friendly matrix operations.

2 Decode Kernel Design

2.1 Problem structure

GDN decode is single-token generation: $T = 1$. Each call updates the recurrent state once and produces one output. Since this is qk4_v8, each q/k head is shared by two V heads. For every batch element and V head, the kernel decays the old state, applies a rank-one correction along the key direction, and then projects the query through the updated state. In these workloads, batch sizes vary from 1 to 64.

2.2 Approach

We started with baseline implementations in both Triton and CUDA. The initial versions used a 2D thread block organized as $32, num\_rows$: the x dimension covered the 128-wide K dimension, while the y dimension selected the state/output row. Effectively, each row in the V tile got its own warp. This mapped naturally to the row-wise state update, but it also meant q/k loads, gate/beta handling, indexing, and reduction setup were repeated across rows.

After a few optimization rounds, we found that CUDA gave us more control over memory access, register use, warp reductions, and instruction scheduling, so we focused on optimizing the kernel

in CUDA. Early versions used shared memory to stage q/k/v data and state rows. Later versions removed most of that staging and switched to direct float4 coalesced loads for state, vectorized bf16 loads for q/k/v, register-resident gate/beta values, and warp-level reductions.

Profiling then showed that the kernel was not memory-bandwidth-bound. Nsight Compute showed low HBM utilization, modest occupancy/SM utilization, and significant scoreboard/dependency stalls.

The main architectural change was moving to a 1D, single-warp block where one warp updates multiple state rows. This amortized q/k loads, gate/beta computation, indexing, and reduction setup across several V rows. In the final versions, we tuned the number of rows per block to balance per-block reuse against wave quantization across B200 SMs, added heavier manual unrolling, and rearranged computations to shorten dependency chains.

3 Prefill Kernel Design

3.1 Problem structure

GDN prefill computes the initial recurrent state and per-token outputs for a batch of prompt sequences. For each token, the algorithm applies gated decay to the previous state, performs a rank-1 update from the token's k and v, and emits an output by projecting the current state with q.

The prefill workloads cover ragged batches with widely varying batch sizes and sequence lengths, from small batches with long sequences, where intra-sequence recurrence dominates, to larger batches with shorter or medium sequences, where there is more parallelism across sequences. Within each sequence, the state update is causal, so token t depends on all earlier tokens in that sequence. The core challenge is preserving this recurrence while exposing parallelism across independent sequences, heads/V-tiles, and chunk-local matrix structure.

3.2 Approach

For prefill, we moved away from a pure token-by-token recurrence. Our first CUDA and Triton versions followed the recurrence directly: they kept a tile of the recurrent state, processed tokens in sequence order, applied gated decay, performed the rank-1 k/v update, and computed the q projection for the output. This was straightforward, but it exposed too little parallelism for long sequences and repeatedly paid the same q/k/gate/reduction costs across V rows.

In CUDA, we first mapped one warp to each V tile and kept state rows in registers, using warp reductions across the 128-wide K dimension. In later versions we introduced chunking, shared-memory staging of Q/K/V and gate values, multi-warp blocks, and eventually WMMA APIs and TMA copies to take advantage of Blackwell chips. We also experimented with warp specialization, and producer-consumer style pipelines. CUDA gave us fine control over loads, reductions, register layout, and scheduling, but the optimized version became very complex and tough to optimize. Even with multiple rounds of optimizations, we could not achieve the same latency as Triton.

In Triton, instead of treating prefill as a long scalar recurrence, we grouped tokens into chunks and rewrote each chunk as structured matrix operations. Inside a chunk, the causal dependence becomes a small lower-triangular system involving KK^T, QK^T, gates, and the incoming state. The resulting strictly lower-triangular structure allows the inverse-like causal correction to be computed using a finite Neumann-style expansion, which we implement with a doubling formulation. This turns much

of the work into dense ‘tl.dot‘ operations over $K = 128$, which maps naturally to tensor cores.

The important improvement was splitting the work into two kernels for the right workloads. The first kernel is a chunk prepass, which computes reusable chunk-local quantities such as gates, cumulative decay, KK^T, QK^T, and inverse-related terms. The second kernel consumes those precomputed terms while walking the sequence state and producing outputs for each V tile. This avoids recomputing the same chunk-local Q/K/gate work independently for every V tile, which is especially valuable when multiple V tiles share the same Q/K structure. We still kept a single-kernel path for smaller cases where launch overhead or extra temporary memory would dominate. The final Triton dispatcher chooses between single-kernel chunkwise execution, split-WY prepass execution, and flat many-sequence variants depending on batch size and average sequence length. It also adapts chunk size and V tile size, uses BF16 tensor-core matmuls for the large contractions, and uses TF32 where the triangular inverse path needs more numerical stability.

4 Performance Analysis

4.1 Decode kernel

Version	Main idea	Mean latency (μs)
v1	2D block baseline: one warp per V row with shared-memory staging	11 us
v2	Remove unnecessary shared-memory staging; use direct vectorized state and q/k/v loads	9.2 us
v3	Single-warp block processes multiple V rows, amortizing q/k loads, gate/beta, indexing, and reductions	6.39 us
v4	Tune V-row tile size across workload buckets to reduce wave quantization and improve small/medium batches	6.35 us
v5	Specialize hot paths with heavier manual unrolling and reordered computation to reduce dependency chains	6.27 us

4.2 Prefill kernel

Stage	Main idea	Mean latency (μs)
Initial mixed dispatch	Started with multiple Triton paths, including a broad fallback for short shapes	145.51
Unified chunkwise	Removed the slow fallback and used chunkwise recurrence as the default path	117.85
Better autotune key	Tuned separately by chunk size and sequence-count regime instead of using one generic schedule	115.07
Split-WY low-N	Added a two-kernel path that precomputes reusable chunk-local terms for long low-batch workloads	113.57
Compact WY workspace	Reduced temporary WY storage and memory traffic for the split path	108.87
Many-sequence Flat-WY	Flattened chunks across sequences to expose more parallelism for larger ragged batches	68.98
High-N flat refinement	Shared Q/K work across grouped V heads and simplified metadata handling in the high-sequence-count path	61.50

5 Tools and Languages

The decode kernel was implemented in CUDA C++ to allow direct control over vectorized memory operations, warp-level reductions, register placement, and instruction ordering. The prefill kernels were implemented in Triton because the final chunkwise formulation maps most of the expensive work to tensor-core matrix operations.

All implementations were validated against the reference PyTorch implementation using the FlashInfer benchmark harness. Correctness checks were run locally and on Modal, while final performance decisions were based on benchmark and profiling runs on Modal B200 GPUs. We used Nsight Compute for profiling CUDA kernels and harness-level latency measurements for comparing kernel variants across the competition workload set.

6 Human vs. Agent Contributions

This was an agent-assisted submission, but the optimization direction was human-led. Human experts defined the GDN reformulation, identified the likely bottlenecks, selected which kernel variants were worth pursuing, and reviewed whether changes were genuine performance improvements rather than benchmark-specific artifacts.

Coding agents were used to accelerate implementation and evaluation. Their main role was to modify active kernel variants, run correctness and benchmark targets through the harness, summarize timing differences, and help explore localized implementation alternatives. In our experience, the agents were most effective when operating inside a constrained harness with a strong human-provided baseline, explicit hypotheses, and objective benchmark feedback.

6.1 AGENTS.md

Our `AGENTS.md` defined the rules and workflow for the agent to drive kernel optimization. It described which files the agent was allowed to edit, how kernel versions should be managed, and what evidence was required before accepting a change. At a high level, we instructed the agent to:

- Edit only the active kernel version, while preserving past versions for comparison.
- Start each experiment with a performance hypothesis.
- Make one focused kernel change at a time, with a short explanation.
- Validate correctness, numerical accuracy, and latency using the fixed harness.
- Accept or reject changes based on timing and profiling evidence.

6.2 Harness

We built a robust evaluation harness around the kernels. It exposed stable Makefile targets for local and Modal correctness checks, Modal benchmarking, and profiling, plus scripts for parsing outputs and comparing latency across kernel versions.

6.3 Autoresearch

Separately, we used a Karpathy-style autoresearch loop for longer autonomous search: edit the active code, run the harness targets, keep improvements, discard regressions, and continue.